

User Manual: Parallelized Gaussian Process Interpolative Decomposition

Aiden Incheol Jung

November 3, 2025

1 Introduction

This manual is a user guide to use Parallelized Gaussian Process Interpolative Decomposition (P-GP-ID) enabling **efficient frequency sweep** for VWT-IE-Solver-Unified-DG.x solver. P-GP-ID employs Gaussian process regression (GPR) with an adaptive sampler to maximize accuracy with minimum number of simulations. Furthermore, the process is parallelized using OpenMPI for efficient computation.

P-GP-ID runs in an iterative manner, adaptively requesting new samples that minimizes the prediction uncertainty. For the samples, the embedded full-wave solver VWT-IE-Solver-Unified-DG.x is called to provide ground truth for the requested sampling point. This process is parallelized and the workflow is illustrated in Fig. 1.

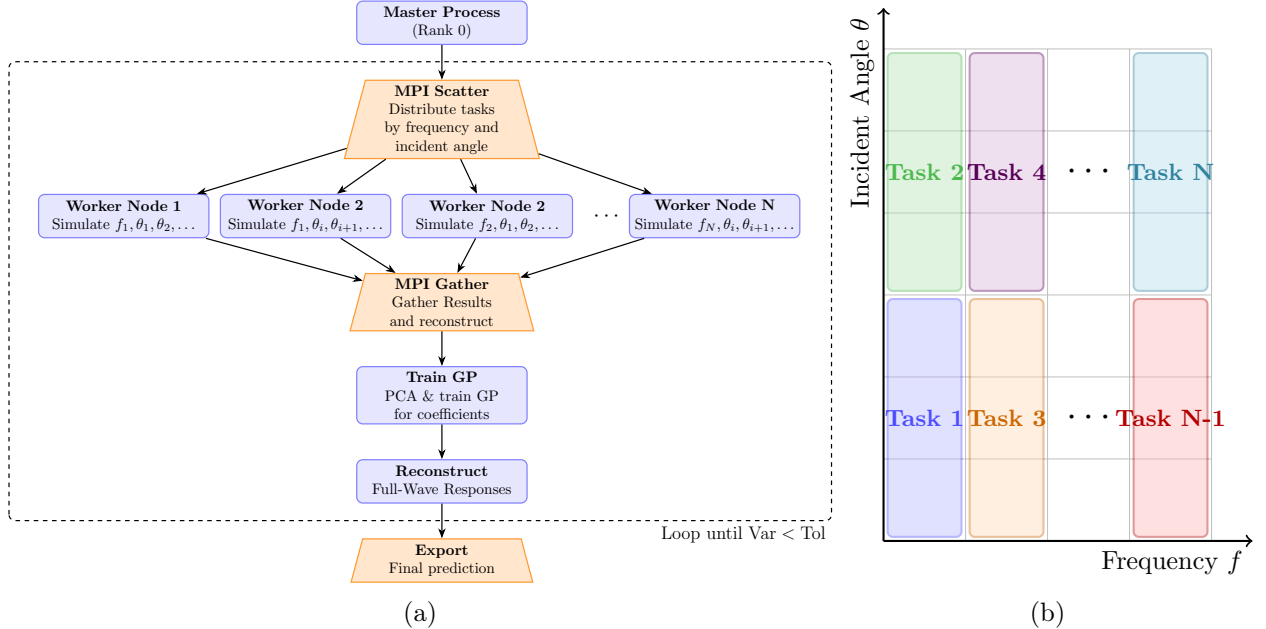


Figure 1: (a) Workflow of P-GP-ID and (b) Task chunking for each node

Since a frequency-domain solver is used, the tasks may naturally be distributed per frequency. However, considering a general cluster computing environment, the number of requested frequency is likely to be smaller than the number of computing nodes. To distribute the task across all the nodes, tasks are chunked in a 2-dimensional (2D) manner: frequency and incident angle.

2 Installation

2.1 pre-requisites

P-GP-ID is written in **Python** which is an interpreter language. Therefore, the user needs to install the python dependencies for the code to run properly:

- Python
- gpytorch
- scikit-learn
- numpy
- scipy
- mpi4py
- matplotlib (To use validation)
- openmpi (the same version in the system)

P-GP-ID also depends on the following system dependencies:

- openmpi (the same version as the Python module)
- rsync (if using a distributed filesystem)

If the user is not willing to use a virtual environment (which is NOT recommended), the packages may be installed in system level. To use a virtual environment, refer to the following section.

```
1 sudo apt install pip
2 pip install gpytorch scikit-learn numpy scipy mpi4py matplotlib openmpi
```

If the user is attempting to run P-GP-ID in a cluster environment using a job scheduler (**SLURM**, etc.; This is the highly likely if the user is an end-user of the cluster), the user may ignore the system dependencies. If the cluster environment is without a job scheduler and using a distributed filesystem, install **rsync** if not installed:

```
1 sudo apt install rsync
```

2.2 Anaconda

When using an interpreter language such as **Python**, to have the system environment intact, it is highly recommended to use a virtual environment. In this user guide, **Anaconda** is used. To install anaconda, refer to **Anaconda**, or **miniconda**.

Once **Anaconda** is installed, a new environment (**myenv** in the example) may be created using the pre-defined environment configuration file:

```
1 conda env create -n myenv -f gpytorch.yaml
```

This is not guranteed to work in any environment since dependencies may differ machine by machine. In such cases, the user may try to create an environment manually:

```
1 conda create -n myenv gpytorch scikit-learn numpy scipy mpi4py matplotlib openmpi
```

2.3 openmpi

P-GP-ID use OpenMPI for parallel computing in a distributed memory environment. However, even if the OpenMPI environment is already successfully established, the use of virtual environment may jeopardize it. The most likely reason of error is version mismatching across nodes. To verify, use the following command on each node for version checking:

```
1 conda run -n myenv python -m mpi4py.run --mpi-lib-version
```

The version must match across all nodes but also their system openmpi version:

```
1 conda deactivate # run the following command outside the virtual environment
2 mpirun --version
```

In case of version mismatching, the user may attempt to install the specific version of openmpi in each virtual environment. However, considering the system-level openmpi is successfully established, the user may use the advantage of it. First, check and store the path to the openmpi installation

```
1 export OMPI_DIR="$(dirname "$(dirname "$(which mpirun)"))"
2 echo $OMPI_DIR
3 ls $OMPI_DIR/bin/mpirun
```

And then re-build mpi4py using the system openmpi as backend:

```
1 conda activate myenv
2 conda install pip # if not installed
3 export MPICC=$OMPI_DIR/bin/mpicc
4 export MPICXX=$OMPI_DIR/bin/mpicxx
5 export PATH=$OMPI_DIR/bin:$PATH
6 export LD_LIBRARY_PATH=$OMPI_DIR/lib:$LD_LIBRARY_PATH
7 # Disable conda's compiler_compat wrappers
8 export _CONDA_PYTHON_SYSCONFIGDATA_NAME=
9 export CONDA_BUILD_SYSROOT=
10 # Force use of system ld
11 export LDFLAGS="-Wl,-rpath,$OMPI_DIR/lib"
12 # Now build mpi4py using system compilers
13 CC=$MPICC CXX=$MPICXX python -m pip install --no-cache-dir --no-binary mpi4py
    mpi4py
```

3 Setting up

3.1 Simulation directory

To allow P-GP-ID to use the full-wave solver VWT-IE-Solver-Unified-DG.x, a simulation directory that contains the simulation inputs needs to be setup. For the detailed information regarding the simulations inputs, refer to the VWT-IE-Solver-Unified-DG.x user manual. Make a directory (default: ./data/VWT-data) that contains the simulation directory and VWT-IE-Solver-Unified-DG.x:

```
data/VWT-data/
├── VWT-IE-Solver-Unified-DG.x
└── simulation/
    ├── model_name.domain
    ├── ...
    ├── subdomain_nameN.tri
    └── subdomain_nameN.rgmatflg
```

This directory must be supplied to P-GP-ID by arguments, the process of which will be depicted in following sections. For future use, the user may add additional directories under `data/VWT-data/`:

```
data/VWT-data/
├── VWT-IE-Solver-Unified-DG.x
├── simulation1/
└── simulation2/
```

All the simulations under the directory `data/VWT-data/` will attempt to run simulations with the executable included in the directory (`data/VWT-data/VWT-IE-Solver-Unified-DG.x`).

3.2 Configuration

P-GP-ID takes three configuration files:

- `config_simulation.cfg`
- `config_gaussian_process.cfg`
- `hosts.txt`

`hosts.txt` is exclusively used for MPI jobs in a distributed filesystem. If the user is using a shared filesystem (most likely unless the user is using an in-house cluster), this file is not necessary.

The other two files provide arguments to P-GP-ID. Configuration files support both Python style (`#`) and C style (`//`) comments. Any lines decorated with the comment keywords are treated as comments, and can be used to label each parameters for readability. Any lines without the decorations are arguments which are supplied to P-GP-ID, and position sensitive. Therefore, they must strictly be in order.

The configurations files will be automatically generated if missing, using the default arguments. The user may use these as templates. Also, while running, P-GP-ID will attempt to overwrite the configuration files with the given arguments. The followings are examples of configuration files.

`config_simulation.cfg`:

```
1 ### Simulation Path ###
2 # default path
3 # The user may use multiple lines but also may mix # and //
4 # If multiple line comments are used, they will be merged into one-line comment
   later.
5 ./data/VWT-data/sphere
6 ### Simulation (Model) Name ###
7 # Automatically detected if None
8 None
9 ### Input Angle Settings ###
10 # [start, end, step] in degrees
11 0 180 1
12 ### Angular Sweep Type (0: phi-sweep, 1: theta-sweep) ###
13 0
14 ### Frequency Settings ###
15 # [start, end, step] in MHz
16 1000 2000 1
17 ### Validate? ###
18 # True for debugging purpose only.
19 # case-insensitive
20 # if true, after the code completes training, the code actually runs frequency
   sweep to compare it with its prediction.
```

```

21 False

config_gaussian_process.cfg:
1 // Acquisition Function Type (0: maximum variance, 1: expected improvement, 2:
  upper confidence bound)
2 0
3 // X normalization Type (0: do not normalize, 1: z-score, 2: min-max, 3: power
  transform, 4: standardized power transform, 5: scaled z-score)
4 // Recommended to choose between 1 or 2
5 1
6 // Number of Initial Samples
7 // This is the number of initial frequencies. The code will initially run
  simulations for them
8 3
9 // Sampling Addition per Iteration
10 // Will be best if the number is a divisor of the number of computing nodes
11 // Will be best not use too large numbers
12 // e.g., [2, 3, 4, 6] when the cluster has 36 computing nodes
13 1
14 // Number of Terms (LF-NSM Kernel Parameter)
15 // increase if the behavior is complicated
16 3
17 // Sampling Strategy (0: grid, 1: latin-hypercube)
18 1
19 // Maximum Iteration for Gaussian Process
20 20
21 // Prediction Energy Tolerance
22 1e-3

```

3.3 hosts.txt (See when using a distributed filesystem)

This subsection is written under the assumption that the system in which P-GP-ID runs is without a job scheduler nor a shared filesystem. Unless the user themselves constructed and maintains the cluster, this section will not be needed: the cluster system should be using both job scheduler such as SLURM and shared filesystem.

Without a job scheduler, the user will have to supply the identity of computing nodes (either in ip address or DNS) either by a file or arguments:

```

1 mpirun --hostfile hosts.txt -n 4 run.sh
2 mpirun --host node01:2,node02:2 -n 4 run.sh

```

where `hosts.txt` contains

```

1 192.168.x.y1 slots=2
2 192.168.x.y2 slots=2

```

In P-GP-ID, when a distributed filesystem is used, it is REQUIRED to use `hosts.txt` file. This attributes to the very nature of the distributed filesystem and the task-chunking approach P-GP-ID use depicted in Fig. 1 (b). Since each task-chunk does not fully cover all queries for the frequency, its output becomes a partial data for the frequency. In distributed filesystem, these partial data are distributed across computing nodes. To merge these fractions, P-GP-ID uses `rsync` to gather all the partial data directories into the `root` node which integrates them into complete data per frequency. Therefore, each node needs to know the the ip address of the root node, which is the destination of the file transmission. To enable this, P-GP-ID attempts to access `hosts.txt` file, assuming that the first line contains the ip address of the root node. If not exist, P-GP-ID will assume that the user is using a shared filesystem thus will not transfer files between computing nodes.

4 Running

4.1 Sequential Run

To run the program using default settings from `.cfg` files:

```
1 conda activate myenv # if using anaconda
2 python main.py
```

The user may overwrite the default settings using arguments. e.g., the following command will make P-GP-ID to use `./data/VWT-data/simulation` as the simulation directory, and will overwrite the corresponding `.cfg` file.

```
1 python main.py -p ./data/VWT-data/simulation
```

To see all available arguments:

```
1 python main.py -h
```

Descriptions on each argument are in the following section.

Following is the list of default arguments:

	Value	Meaning
-p	./data/VWT-data/sphere	simulation directory
-m		model name (*.domain) is automatically found
-A	0 180 1	angular window will be set to [0, 180] DEG with 1 DEG interval
-S	0	phi-sweep
-F	1000 2000 1	frequency band will be set to [1000, 2000] MHz with 1 MHz interval
-v		no validation
-a	0	acquisition function: maximum variance
-x	1	x-data normalization strategy: z-score (standard normalization)
-n	3	number of initial frequency samples
-add	1	added frequency samples per iteration
-t	3	number of terms used for Gaussian process kernel
-s	1	sampling strategy: Latin Hypercube Sampling
-i	20	maximum iteration for Gaussian Process
-T	5e-5	IVR tolerance; tolerance for Gaussian Process

4.2 Parallel Run

To run the program using MPI using 4 nodes:

```
1 mpirun -n 4 python main.py
```

When using Anaconda, the virtual environment needs to be activated on each computing node. The user may create a bash file:

```
1 #!/bin/bash file: run_node.sh
2 unset DISPLAY # disables GNOME
3 export OMP_NUM_THREADS=$(nproc) # allow each node to use maximum cores
4 source $(conda info --base)/etc/profile.d/conda.sh # enables anaconda
5 conda activate myenv # activates the virtual environment
6 python main.py # runs P-GP-ID
```

and then run the bash code:

```
1 mpirun -n 4 run_node.sh
```

5 Arguments

This section dedicates to explain each argument given to P-GP-ID.

- **-p, -path-simulation:**
Used to specify the simulation directory.
- **-m, -model-name:**
It is the name of the simulation under the simulation directory, which is the name of `.domain` file. For instance, if user has set up a simulation under `sim` directory with `model.domain` under the directory, then the correct argument is “model.” If not provided, the code will attempt to find `.domain` file under the simulation directory. If there is more than one `.domain` file, the automatic search fails with a **Runtime Error**.
- **-A, -angle, -angle-settings:**
Requires 3 arguments: [start, end, step] of angles to sweep. For example, [0, 100, 25] will create [0, 25, 50, 75, 100]
- **-S, -ast, -angular-sweep-type:**
Type of angular sweep. Currently, the code relies on a single angular sweep either ϕ - or θ -sweep. 0: ϕ -sweep, 1: θ -sweep
- **-F, -freq, -frequency-settings:**
Requires 3 arguments: [start, end, step] of frequencies. For example, [0, 180, 60] will create [0, 60, 120, 180]. The predicted farfield data for the corresponding frequencies will be exported to EFAR directory. The total number of frequencies will also be used as the number of candidates for adaptive sampling process, thus it is recommended not to use too coarse sampling.
- **-v, -no-validate:**
For Debugging Purpose. Flag to validate. At the end of all process, P-GP-ID will attempt to validate the accuracy by calculating RMSE of the prediction and ground-truth for grid-sampled inputs. This means, if the corresponding grid-sampled ground-truth is not prepared, the code will attempt to run simulations over the entire validation points.
- **-a, -ax-fx, -acquisition-function:**
Type of acquisition function for adaptive sampling. An integer $\in [0, 2]$ can be given. User can choose between [maximum variance, expected improvement, upper confidence bound]. Defaults to 0.
- **-n, -n-init:**
Initial number of frequency sampling. The initial samples are always sampled with Latin Hypercube Sampling. For efficient simulation, the number is recommended to be chosen as a divisor of the number of the number of computing nodes. Defaults to 3.
- **-add, -freq-add:**
Number of added frequency samples per Gaussian Process iteration. Defaults to 1. For efficient simulation, the number is recommended to be chosen as a divisor of the number of the number of computing nodes, and not to be too large. e.g., when the cluster system has 36 nodes, a good candidate for `-add` argument will be [1, 2, 3, 4, 6].
- **-t, -terms:**
Terms. Currently stacked RBFP kernel is used, where terms will define the number of stacks. Larger number will be beneficial for the GPR engine to capture complex spectral behavior but will increase computational complexity as well. Defaults to 3.

- **-s, -sample, -sampling-strategy:**
Type of sampling strategy. 0 or 1 can be given. User can choose between [Grid Sampling, Latin Hypercube Sampling]. Grid Sampling has higher chance to encounter a `RuntimeError` in adaptive sampling. Defaults to 1.
- **-i, -max-iter:**
Maximum iteration for Adaptive Sampler. Therefore, the maximum frequency samples will be determined as $\boxed{-n} + \boxed{-add} * \boxed{-i}$.
- **-T, -tol:**
Tolerance for integrated variance reduction (normalized IVR; normalized weighted sum of variances over basis). Defaults to 1e5.